

Regularisation

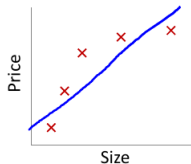
Assit.Prof.Dr.Emre Özbilge

Artificial Intelligence Engineering Department
Faculty of Engineering
Cyprus International University

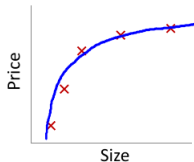
March 15, 2024

The Problem of Overfitting I

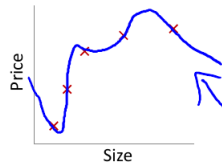
Example: Linear regression (housing prices)



$\rightarrow \theta_0 + \theta_1 x$
"Underfit" "High bias"



$\rightarrow \theta_0 + \theta_1 x + \theta_2 x^2$
"Just right"

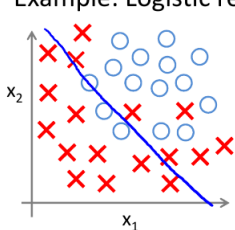


$\rightarrow \theta_0 + \theta_1 x + \theta_2 x^2 + \theta_3 x^3 + \theta_4 x^4$
"Overfit" "High variance"

Overfitting: If we have too many features, the learned hypothesis may fit the training set very well ($J(\theta) = \frac{1}{2m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)})^2 \approx 0$), but fail to generalize to new examples (predict prices on new examples).

The Problem of Overfitting II

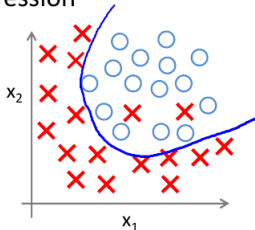
Example: Logistic regression



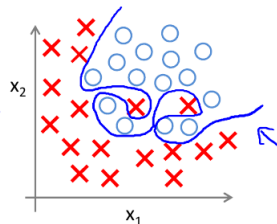
$$\rightarrow h_{\theta}(x) = g(\theta_0 + \theta_1 x_1 + \theta_2 x_2)$$

(g = sigmoid function)

"Underfit"



$$g(\theta_0 + \theta_1 x_1 + \theta_2 x_2 + \theta_3 x_1^2 + \theta_4 x_2^2 + \theta_5 x_1 x_2)$$



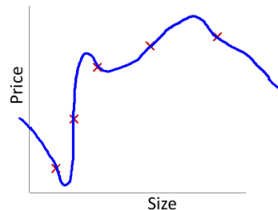
$$g(\theta_0 + \theta_1 x_1 + \theta_2 x_1^2 + \theta_3 x_1^2 x_2 + \theta_4 x_1^2 x_2^2 + \theta_5 x_1^2 x_2^3 + \theta_6 x_1^3 x_2 + \dots)$$

"Overfit"

The Problem of Overfitting III

Addressing overfitting:

x_1 = size of house
 x_2 = no. of bedrooms
 x_3 = no. of floors
 x_4 = age of house
 x_5 = average income in neighborhood
 x_6 = kitchen size
:
:
 x_{100}



The Problem of Overfitting IV

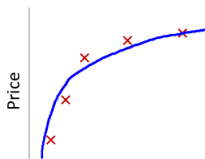
Addressing overfitting:

Options:

1. Reduce number of features.
 - — Manually select which features to keep.
 - — Model selection algorithm
2. Regularization.
 - — Keep all the features, but reduce magnitude/values of parameters θ_j .
 - Works well when we have a lot of features, each of which contributes a bit to predicting y .

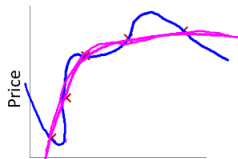
Cost Function I

Intuition



Size of house

$$\theta_0 + \theta_1 x + \theta_2 x^2$$



Size of house

$$\theta_0 + \theta_1 x + \theta_2 x^2 + \cancel{\theta_3 x^3} + \cancel{\theta_4 x^4}$$

↑ ↑

Suppose we penalize and make θ_3, θ_4 really small.

$$\rightarrow \min_{\theta} \frac{1}{2m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)})^2 + 1000 \underline{\theta_3^2} + 1000 \underline{\theta_4^2}$$

$\underline{\theta_3 \approx 0} \quad \underline{\theta_4 \approx 0}$

Cost Function II

Regularization.

Small values for parameters $\theta_0, \theta_1, \dots, \theta_n$

- “Simpler” hypothesis
- Less prone to overfitting

θ_2, θ_4
 ≈ 0

Housing:

- Features: $x_1, x_2, \dots, x_{100}$
- Parameters: $\theta_0, \theta_1, \theta_2, \dots, \theta_{100}$

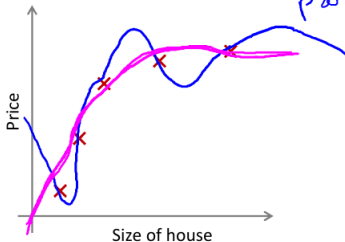
$$J(\theta) = \frac{1}{2m} \left[\sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)})^2 + \lambda \sum_{j=1}^n \theta_j^2 \right]$$

~~$\theta_1, \theta_2, \theta_3, \dots, \theta_{100}$~~ ~~$\lambda$~~

Cost Function III

Regularization.

$$\min_{\theta} J(\theta) = \frac{1}{2m} \left[\underbrace{\sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)})^2}_{\text{data fit}} + \underbrace{\lambda \sum_{j=1}^n \theta_j^2}_{\text{regularization parameter}} \right]$$



Cost Function IV

In regularized linear regression, we choose θ to minimize

$$J(\theta) = \frac{1}{2m} \left[\sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)})^2 + \lambda \sum_{j=1}^n \theta_j^2 \right]$$

What if λ is set to an extremely large value (perhaps far too large for our problem, say $\lambda = 10^{10}$)?

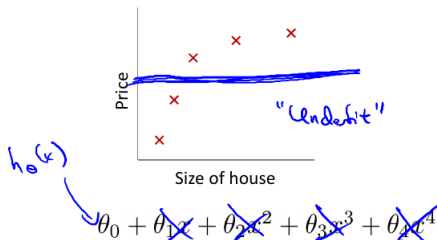
- Algorithm works fine; setting λ to be very large can't hurt it
- Algorithm fails to eliminate overfitting.
- Algorithm results in underfitting. (Fails to fit even training data well).
- Gradient descent will fail to converge.

Cost Function V

In regularized linear regression, we choose θ to minimize

$$J(\theta) = \frac{1}{2m} \left[\sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)})^2 + \lambda \sum_{j=1}^n \theta_j^2 \right]$$

What if λ is set to an extremely large value (perhaps far too large for our problem, say $\lambda = 10^{10}$)?



$$\begin{aligned} \theta_1, \theta_2, \theta_3, \theta_4 \\ \theta_1 \approx 0, \theta_2 \approx 0 \\ \theta_3 \approx 0, \theta_4 \approx 0 \end{aligned}$$

$h_{\theta}(x) = \theta_0$

L1 and L2 Regularisation Methods I

- ▶ The key difference between these techniques is that L1 shrinks the less important feature's coefficient to zero thus, removing some feature altogether. So, this works well for feature selection in case we have a huge number of features.

1. L1 Regularisation adds “absolute value of magnitude” of coefficient as penalty term to the loss function.

$$J(\theta) = \frac{1}{2m} \left[\sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)})^2 + \lambda \sum_{j=1}^n |\theta_j| \right]$$

2. L2 Regularisation adds “squared magnitude” of coefficient as penalty term to the loss function.

$$J(\theta) = \frac{1}{2m} \left[\sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)})^2 + \lambda \sum_{j=1}^n \theta_j^2 \right]$$

Regularised Linear Regression I

Regularized linear regression

$$J(\theta) = \frac{1}{2m} \left[\sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)})^2 + \lambda \sum_{j=1}^n \theta_j^2 \right]$$

$$\min_{\theta} J(\theta)$$



Regularised Linear Regression II

Gradient descent

Repeat {

$$\rightarrow \theta_0 := \theta_0 - \alpha \frac{1}{m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)}) x_0^{(i)}$$

$$\theta_j := \theta_j - \alpha \left[\frac{1}{m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)}) x_j^{(i)} - \frac{\lambda}{m} \theta_j \right] \quad (j = \cancel{x}, 1, 2, 3, \dots, n)$$

}

$$\theta_j := \theta_j (1 - \alpha \frac{\lambda}{m}) - \alpha \frac{1}{m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)}) x_j^{(i)}$$

$$1 - \alpha \frac{\lambda}{m} < 1$$

$$0.99$$

$$\theta_j \times 0.99$$

$$\theta_j^2$$

$$\theta_0$$

$$\theta_1, \theta_2, \dots, \theta_n$$

$$\frac{\partial}{\partial \theta_0} J(\theta)$$

$$J(\theta)$$

Regularised Linear Regression III

Normal equation

$$\begin{aligned}
 \underline{X} &= \begin{bmatrix} (x^{(1)})^T \\ \vdots \\ (x^{(m)})^T \end{bmatrix} \quad \leftarrow \text{m} \times (\text{n}+1) \\
 \underline{y} &= \begin{bmatrix} y^{(1)} \\ \vdots \\ y^{(m)} \end{bmatrix} \quad \mathbb{R}^m \\
 &\rightarrow \min_{\theta} J(\theta) \\
 &\rightarrow \Theta = \left(X^T X + \lambda \underbrace{\begin{bmatrix} 0 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}}_{(n+1) \times (n+1)} \right)^{-1} X^T y
 \end{aligned}$$

$\frac{2}{2\theta_j} J(\theta) \stackrel{\text{set}}{=} 0$

$\in \mathbb{R}^n, n=2$

Regularised Linear Regression IV

Non-invertibility (optional/advanced).

Suppose $m \leq n$, $\leftarrow$
(#examples) (#features)

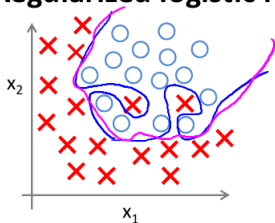
$$\theta = \underbrace{(X^T X)^{-1}}_{\text{non-invertible / singular}} X^T y \quad \text{pinv} \quad \frac{\text{inv}}{\lambda}$$

If $\lambda > 0$,

$$\theta = \left(\underbrace{X^T X + \lambda \begin{bmatrix} 0 & & & \\ & 1 & & \\ & & 1 & \\ & & & \ddots \\ & & & & 1 \end{bmatrix}}_{\text{invertible.}} \right)^{-1} X^T y$$

Regularised Logistic Regression I

Regularized logistic regression.



$$h_{\theta}(x) = g(\theta_0 + \theta_1 x_1 + \theta_2 x_1^2 + \theta_3 x_1^2 x_2 + \theta_4 x_1^2 x_2^2 + \theta_5 x_1^2 x_2^3 + \dots)$$

Cost function:

$$\rightarrow J(\theta) = - \left[\frac{1}{m} \sum_{i=1}^m y^{(i)} \log h_{\theta}(x^{(i)}) + (1 - y^{(i)}) \log(1 - h_{\theta}(x^{(i)})) \right] + \frac{\lambda}{2m} \sum_{j=1}^n \theta_j^2$$

$\theta_1, \theta_2, \dots, \theta_n$

Regularised Logistic Regression II

Gradient descent

Repeat {

$$\rightarrow \theta_0 := \theta_0 - \alpha \frac{1}{m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)}) x_0^{(i)}$$

$$\rightarrow \theta_j := \theta_j - \alpha \left[\frac{1}{m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)}) x_j^{(i)} - \frac{\lambda}{m} \theta_j \right] \leftarrow$$

$(j = \text{red X}, 1, 2, 3, \dots, n)$
 $\theta_1, \dots, \theta_n$

}

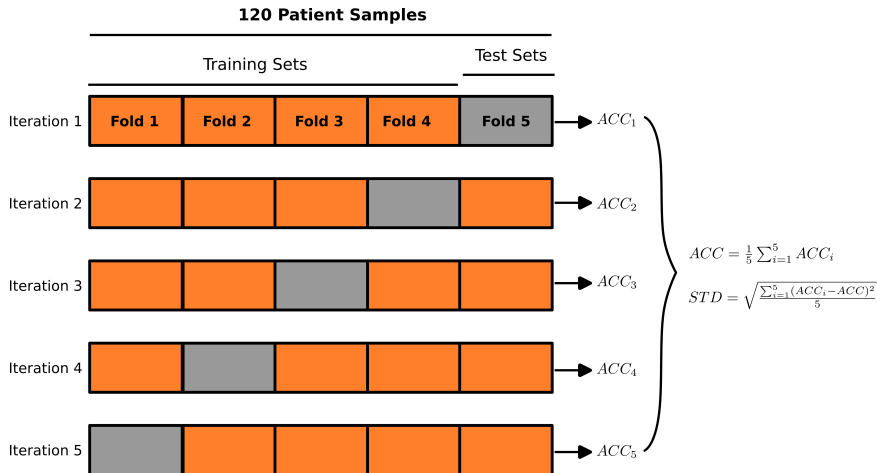
$$\frac{2}{\partial \theta_j} \frac{\mathcal{J}(\theta)}$$

$$\underline{h_{\theta}(x)} = \frac{1}{1 + e^{-\theta^T x}}$$

Cross-validation (CV): Find optimal hyper-parameters

1. `lam_vec = [0;0.0005;0.001;0.003;0.01;0.03;0.1;0.3;1;3;10];`
2. `lr_vec = [0.0007;0.0005;0.0001;0.007;0.005;0.001;0.03;0.01;0.1;0.3];`
3. `error_val = zeros(length(lam_vec), length(lr_vec));`
4. `max_epoch_CV = 100`
5. `max_epoch_final = 1,000`
6. **for** `i = 1:length(lam_vec)`
 - 6.1 **for** `j = 1:length(lr_vec)`
 - 6.1.1 `theta = trainLinearReg(Xtrain, ytrain, lam_vec(i), lr_vec(j), Xval, yval, max_epoch_CV);`
 - 6.1.2 `error_val(i,j) = linearRegCostFunction(Xval, yval, theta);`
 - 6.2 **end**
7. **end**
8. `best_lam_idx, best_lr_idx = min(error_val);`
9. `theta = trainLinearReg(Xtrain, ytrain, lam_vec(best_lam_idx), lr_vec(best_lr_idx), Xval, yval, max_epoch_final);`
10. `error_test, y_test_pred = linearRegCostFunction(Xtest, ytest, theta);`

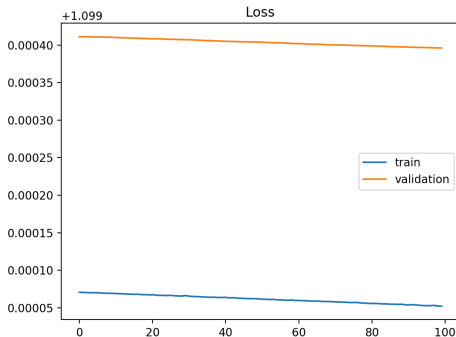
K-Fold Cross-validation



Underfit Learning Curves

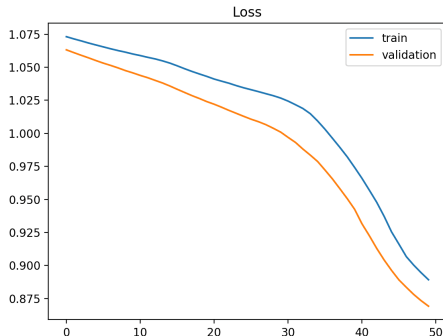
- ▶ Underfitting refers to a model that cannot learn the training dataset.
- ▶ An underfit model can be identified from the learning curve of the training loss only.
- ▶ It may show a flat line or noisy values of relatively high loss, indicating that the model was unable to learn the training dataset at all.
- ▶ An example of this is provided below and is common when the model does not have a suitable capacity for the complexity of the dataset.

Learning Curves to Diagnose Model Performance II



- ▶ An underfit model may also be identified by a training loss that is decreasing and continues to decrease at the end of the plot.
- ▶ This indicates that the model is capable of further learning and possible further improvements and that the training process was halted prematurely.

Learning Curves to Diagnose Model Performance III

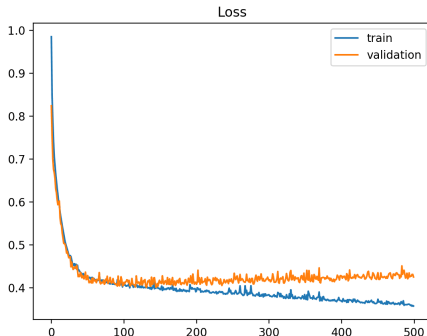


- ▶ A plot of learning curves shows underfitting if:
 - The training loss remains flat regardless of training.
 - The training loss continues to decrease until the end of training.

Overfit Learning Curves

- ▶ Overfitting refers to a model that has learned the training dataset too well, including the statistical noise or random fluctuations in the training dataset.
- ▶ The problem with overfitting, is that the more specialized the model becomes to training data, the less well it is able to generalize to new data, resulting in an increase in generalization error. This increase in generalization error can be measured by the performance of the model on the validation dataset.
- ▶ This often occurs if the model has more capacity than is required for the problem, and, in turn, too much flexibility. It can also occur if the model is trained for too long.

Learning Curves to Diagnose Model Performance V

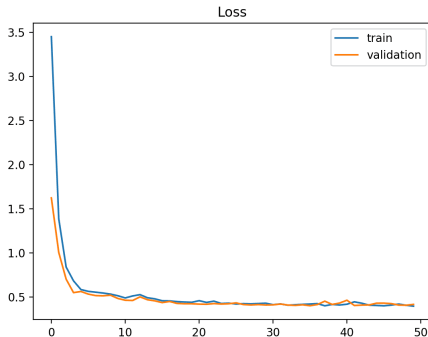


- A plot of learning curves shows overfitting if:
- The plot of training loss continues to decrease with experience.
 - The plot of validation loss decreases to a point and begins increasing again.

Good Fit Learning Curves

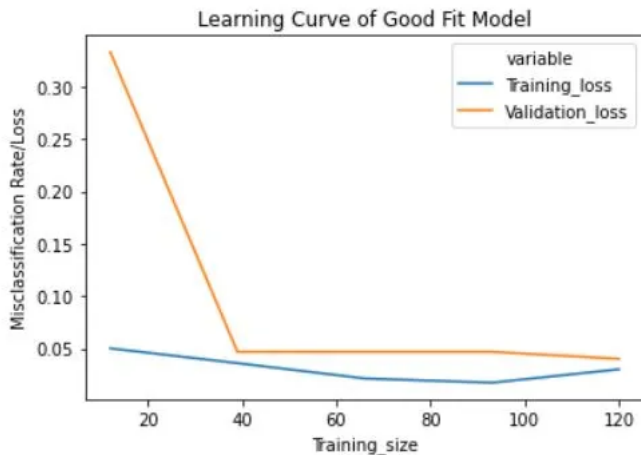
- ▶ A good fit is the goal of the learning algorithm and exists between an overfit and underfit model.
- ▶ A good fit is identified by a training and validation loss that decreases to a point of stability with a minimal gap between the two final loss values.
- ▶ The loss of the model will almost always be lower on the training dataset than the validation dataset. This means that we should expect some gap between the train and validation loss learning curves. This gap is referred to as the “generalization gap”.
- ▶ Continued training of a good fit will likely lead to an overfit.
- ▶ A plot of learning curves shows a good fit if:

Learning Curves to Diagnose Model Performance VII

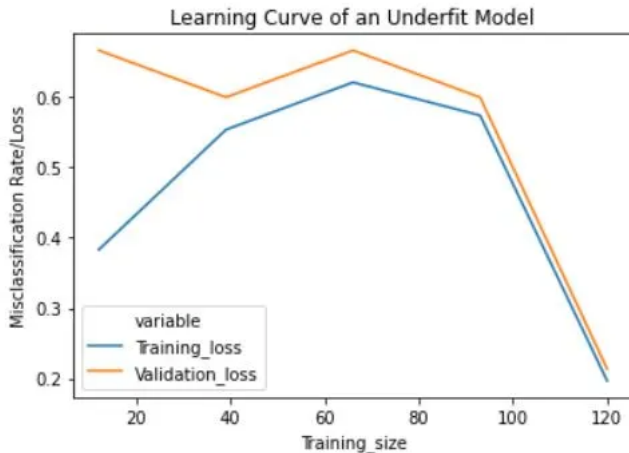


- The plot of training loss decreases to a point of stability.
- The plot of validation loss decreases to a point of stability and has a small gap with the training loss.

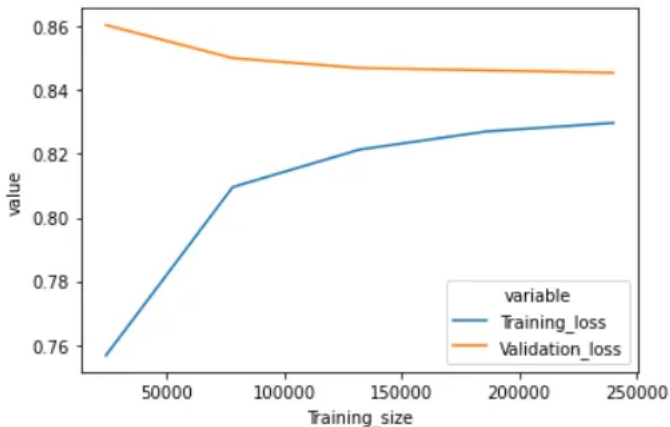
Case study: Diagnose the models I



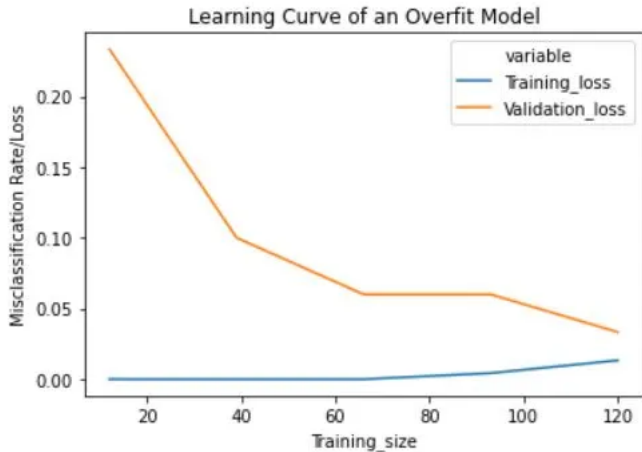
Case study: Diagnose the models II



Case study: Diagnose the models III



Case study: Diagnose the models IV



► Reasons for Underfitting:

1. The model is too simple.
2. The input features is not relevant to represent the target outputs.
3. The size of the training dataset used is not enough.
4. Very large regularization are used to prevent the overfitting.
5. Features are not scaled.

► Reduce Underfitting:

1. Increase model complexity.
2. Increase the number of features, performing feature engineering.
3. Remove outliers from the data.
4. Increase the number of epochs.

► Reasons for Overfitting:

1. High variance and low bias.
2. The model is too complex.
3. The size of the training data.

► Reduce Overfitting:

1. Improving the quality of training data.
2. Increase the training data.
3. Reduce model complexity.
4. Early stopping during the training process.
5. Use Regularization.
6. Use dropout for (deep) neural networks.